

Tabular Reinforcement Learning op Taxi-v3

Een vergelijking van Q-learning en SARSA tegen baselines

Karol Gackowski

Joep Houweling

4 mei 2026

Samenvatting

We onderzoeken in welke mate twee klassieke tabulaire Reinforcement Learning algoritmen — Q-learning en SARSA — de Taxi-v3 omgeving uit Gymnasium kunnen oplossen. Beide algoritmen zijn vanaf nul geïmplementeerd in NumPy, getraind op 3000 episodes en vergeleken met een uniform-random baseline en een Manhattan-shortest-path heuristiek die geen kennis heeft van muren. Q-learning bereikt een gemiddeld rendement van +8,33 met 100% succesvolle leveringen; SARSA −2,32 met 95% succes. Beide verslaan ruimschoots de heuristiek (−156, 21%) en de random baseline (−748, 6%). Hyperparameter-sweeps over α , γ en het ϵ -decay-schema laten zien dat Q-learning robuust is voor α en γ , maar gevoelig is voor te trage ϵ -decay, terwijl SARSA bij hoge α catastrofaal instort vanwege de on-policy bootstrap. We koppelen deze bevindingen terug aan de theorie en bespreken de beperkingen van tabulaire methoden.

1 Introductie

Reinforcement Learning (RL) bestudeert hoe een autonome agent door interactie met een omgeving een gedragsstrategie kan leren die de verwachte cumulatieve beloning maximaliseert [1]. In tegenstelling tot supervised learning is er geen vooraf gegeven dataset met correcte acties; de agent ontdekt zelf welke acties op de lange termijn lonend zijn via trial-and-error.

Voor deze portfolio-opdracht hebben wij gekozen voor de *Taxi* omgeving, oorspronkelijk geformaliseerd door [4] en tegenwoordig opgenomen als **Taxi-v3** in de Gymnasium-suite [5]. De agent bestuurt een taxi op een 5×5 raster met muren, moet een passagier ophalen op één van vier vaste lokaties en deze afleveren op een eveneens vooraf bepaalde bestemming. Elke stap kost één beloningseenheid; een illegale pickup of dropoff kost tien; een succesvolle aflevering levert twintig op.

Waarom RL geschikt is. Drie eigenschappen maken Taxi-v3 een natuurlijk doelwit voor RL en ongeschikt voor supervised of regelgebaseerde alternatieven:

1. **Geen gelabelde dataset.** Er bestaat geen verzameling “correcte” acties per toestand om uit te leren; de agent moet zelf experimenteren.
2. **Lange-termijn credit-assignment.** De +20 beloning bij aflevering moet teruggekoppeld worden naar de tientallen acties die er aan vooraf gingen. Een hand-gecodeerde Manhattan-heuristiek kan dit niet, want zij “ziet” geen muren en raakt vaak vast (zie sectie 4).
3. **Gestructureerde sub-taken.** De optimale strategie hangt af van of de passagier al in de taxi zit. Een statische heuristiek moet deze logica expliciet coderen; RL leert dit impliciet via de toestandsafhankelijkheid van $Q(s, a)$.

Onderzoeksvragen. Dit rapport beantwoordt: (i) hoe presteren Q-learning en SARSA, vanuit nul geïmplementeerd, op Taxi-v3 ten opzichte van een random- en een heuristische baseline? (ii) hoe gevoelig zijn beide algoritmen voor de keuze van α , γ en het ε -decay-schema? (iii) welke kwalitatieve verschillen tussen on-policy (SARSA) en off-policy (Q-learning) leren observeren we in dit domein?

2 Probleemdefinitie en MDP

Taxi-v3 wordt gemodelleerd als een Markov Decision Process (MDP) $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ met de volgende elementen:

Toestandruimte $\mathcal{S}$. $|\mathcal{S}| = 500$ discrete toestanden, gecodeerd als

$$s = ((r \cdot 5 + c) \cdot 5 + p) \cdot 4 + d, \quad (1)$$

waarbij $r, c \in \{0, \dots, 4\}$ de positie van de taxi zijn, $p \in \{0, \dots, 4\}$ de locatie van de passagier (0=R, 1=G, 2=Y, 3=B, 4=in_taxi) en $d \in \{0, \dots, 3\}$ de bestemming.

Actieruimte $\mathcal{A}$. $|\mathcal{A}| = 6$: {south, north, east, west, pickup, dropoff}. Bewegingsacties die tegen een muur of de rand bewegen, laten de agent op zijn plaats. Pickup en dropoff buiten een geldige locatie zijn toegestaan maar kosten beloning.

Beloningsfunctie R . -1 per stap, -10 voor een illegale pickup of dropoff, $+20$ voor een succesvolle aflevering (terminale beloning).

Discount γ . We gebruiken $\gamma = 0,99$ als basis. Omdat de episodes terminaal zijn na succesvolle aflevering, is γ vooral een knop voor de “urgentie” waarmee de agent wil dat de aflevering snel gebeurt.

3 Methode

3.1 Algoritmen

Q-learning (off-policy) [2] schat de actie-waardefunctie $Q(s, a)$ met de update-regel

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]. \quad (2)$$

Het bootstrap-target gebruikt de *greedy* actie in s_{t+1} , ongeacht welke actie de gedragspolitiek (hier ε -greedy) werkelijk gaat selecteren. Daarmee leert Q-learning de optimale greedy-policy, ook als de gedragspolitiek nog veel exploratie bevat.

SARSA (on-policy) [3] bootstrapt met de actie die de gedragspolitiek *daadwerkelijk* selecteert:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]. \quad (3)$$

Daardoor incorporeert SARSA de exploratie-ruis in zijn waardeschatting: de geleerde policy is optimaal *gegeven* dat de gedragspolitiek ε -greedy blijft.

Implementatie. Beide algoritmen zijn als puur Python geïmplementeerd, met de Q-tabel als een $|\mathcal{S}| \times |\mathcal{A}|$ NumPy-array. We gebruiken ε -greedy actie-selectie met willekeurige tie-breaking in de greedy stap, en lineaire ε -decay van 1,0 naar 0,05 over 2000 episodes. Geen enkele RL-bibliotheek (Stable-Baselines, RLlib, e.d.) is gebruikt; we voldoen daarmee aan de eis dat de algoritmen vanaf nul geïmplementeerd zijn.

3.2 Baselines

Random. Uniforme keuze uit de zes acties. Dient als ondergrens.

Heuristiek. Een Manhattan-shortest-path strategie die geen kennis heeft van muren:

- Als de passagier nog niet in de taxi zit: ga naar de pickup-tegel via de actie die de Manhattan-afstand het meest verkleint; voer **pickup** uit zodra je er bent.
- Anders: ga naar de bestemmingstegel via dezelfde regel; voer **dropoff** uit zodra je er bent.

Door de muren niet te kennen, loopt de agent regelmatig vast.

3.3 Hyperparameter-experimenten

We voeren vier sweeps uit, telkens met vijf seeds per waarde en 3000 episodes per run:

1. **Q-learning**, $\alpha \in \{0,1,0,3,0,5,0,9\}$ — gevoeligheid voor de leersnelheid.
2. **Q-learning**, $\gamma \in \{0,8,0,9,0,95,0,99\}$ — gevoeligheid voor de horizon.
3. **Q-learning**, ε -decay-episodes $\in \{200,1000,2000,5000\}$ — gevoeligheid voor het exploratie-schema.
4. **SARSA**, $\alpha \in \{0,1,0,3,0,5,0,9\}$ — spiegel van sweep 1, om on-policy gedrag te isoleren.

3.4 Evaluatieprotocol

Na training worden agenten geëvalueerd over 100 episodes met $\varepsilon = 0$ (puur greedy) en met seeds die niet overlappen met de trainingsseeds. We rapporteren gemiddeld rendement, gemiddelde episodelengte, gemiddeld aantal illegale acties en het afleveringspercentage.

4 Resultaten

4.1 Hoofdvergelijking

Tabel 1 toont de greedy-evaluatie van alle vier de agenten na 3000 trainingsepisodes. Beide RL-algoritmen verslaan de baselines ruimschoots; Q-learning bereikt de optimale prestatie (100% succes, gemiddelde episode-lengte ≈ 13 , in lijn met het theoretische optimum van ongeveer 12–14 stappen per episode), terwijl SARSA er net onder zit met 95% succes. De wandblinde heuristiek presteert nauwelijks beter dan random ondanks haar doelgerichte ontwerp: zonder muurbewustzijn raakt zij in 79% van de episodes verstrikt en haalt zij de tijdslimiet.

Tabel 1: Greedy evaluatie ($\varepsilon = 0$) over 100 episodes. “Mean return” is het gemiddelde gediscounteerde rendement; “Length” is de gemiddelde episodelengte; “Success” is het percentage episodes dat eindigt in een succesvolle aflevering.

Agent	Mean return	Std	Mean length	Success rate
Random	−748,47	130,67	193,9	6%
Heuristiek	−155,74	85,86	160,2	21%
SARSA	− 2,32	45,45	22,3	95%
Q-learning	+ 8,33	2,82	12,7	100%

4.2 Trainingscurves

Figuur 1 laat de leercurves van Q-learning en SARSA zien. Beide algoritmen vertonen het kenmerkende “hockey-stick” patroon: een lange initiële plateau waarin random exploratie weinig positief signaal oplevert, gevolgd door een snelle stijging zodra de agent de eerste succesvolle leveringen ervaart en het +20 signaal zich door de Q-tabel verspreidt. Q-learning convergeert iets sneller en eindigt op een hoger plateau dan SARSA, in lijn met het verschil in greedy-evaluatie uit Tabel 1.



Figuur 1: Trainingscurves (rolling mean over 20 episodes) voor Q-learning en SARSA op Taxi-v3. De stippellijn geeft de theoretisch benaderde optimale return ($\approx +8$).

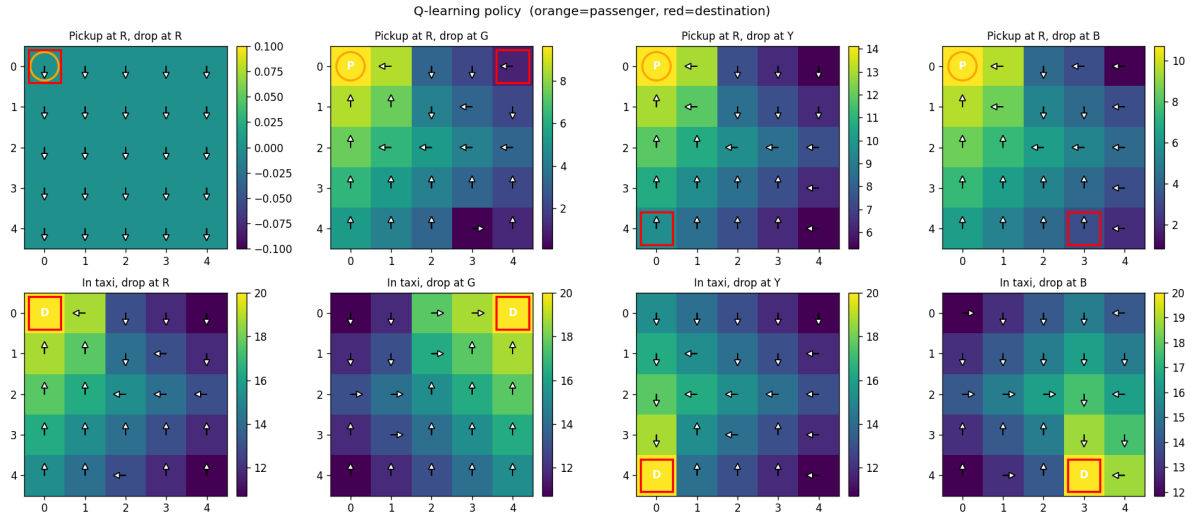
4.3 Geleerde policies

Figuren 2 en 3 visualiseren de greedy policies in acht (passagierslocatie, bestemming) configuraties. Voor elke configuratie tonen we $V(s) = \max_a Q(s, a)$ als heatmap met de greedy actie als pijl per cel; P en D markeren respectievelijk pickup- en dropoff-acties. De donkergroene gradient loopt logisch naar de oranje (passagier) of rode (bestemming) markering. De Q-learning policy is over alle scenario's stabiel coherent; SARSA's policy is grotendeels gelijk maar bevat enkele cellen met een suboptimale actie, wat het lagere succespercentage in Tabel 1 verklaart.

4.4 Hyperparameter-sweeps

Tabel 2 vat de gemiddelde return over de laatste 200 training-episodes samen voor elke sweep, gemiddeld over vijf seeds. Figuren 4, 5, 6 en 7 tonen de bijbehorende leercurves met $\pm 1\sigma$ banden.

Q-learning, α (Fig. 4). Q-learning is opvallend ongevoelig voor de leersnelheid: alle vier de waarden eindigen binnen $\pm 0,3$ van elkaar. Dit is consistent met de theoretische convergentiegarantie van Q-learning onder Robbins–Monro voorwaarden [1]; in de praktijk geldt dat zolang α niet zo extreem laag is dat updates verwaarloosbaar worden, de eindwaardefunctie convergeert naar dezelfde fixed point.



Figuur 2: Q-learning greedy policy in acht scenario's. Bovenste rij: passagier wacht bij **R** (oranje cirkel). Onderste rij: passagier zit al in de taxi. Kolommen: bestemming **R**, **G**, **Y**, **B** (rood vierkant).

Q-learning, γ (Fig. 5). Vergelijkbaar robuust over de geteste range. Voor Taxi-v3 zijn episodes relatief kort (typisch < 20 stappen na convergentie), waardoor het verschil tussen $\gamma = 0,8$ en $\gamma = 0,99$ in de praktijk klein blijft.

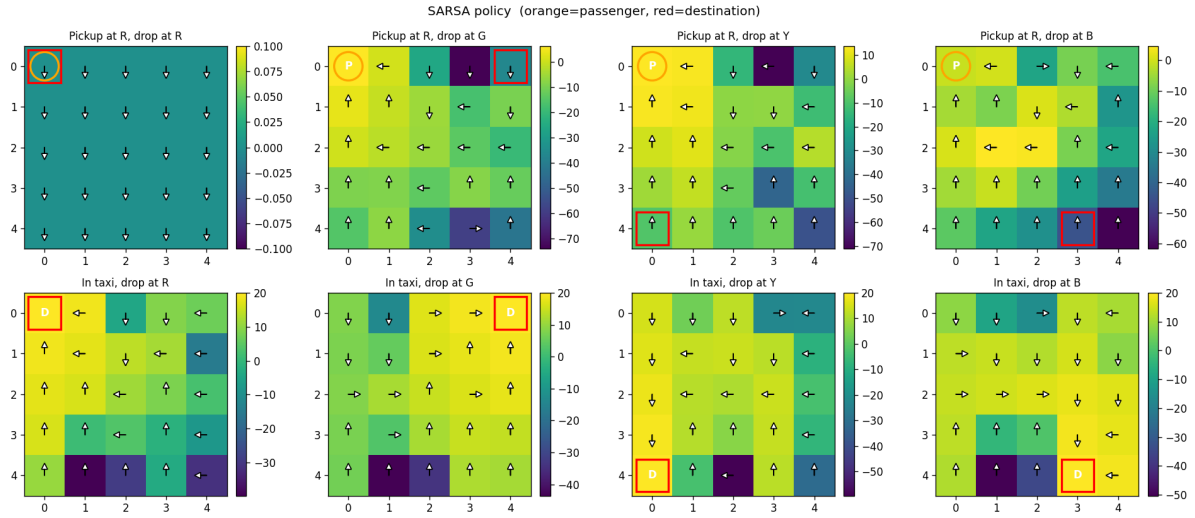
Q-learning, ϵ -decay (Fig. 6). Hier vinden we de eerste duidelijke negatieve uitslag: bij een ϵ -decay-window van 5000 episodes is ϵ na 3000 training-episodes nog ruim boven 0,3, zodat een aanzienlijk deel van de *evaluatie*-stappen in de laatste 200 episodes nog willekeurig zijn. De return zakt dan naar $-35,27$. Dit illustreert een belangrijk praktisch punt: ook al levert Q-learning een correcte greedy-policy op, de gemeten trainings-return wordt sterk beïnvloed door de gedragspolitiek.

SARSA, α (Fig. 7). Hier vinden we de meest dramatische bevinding: SARSA is veel gevoeliger voor α dan Q-learning. Bij $\alpha = 0,9$ stort SARSA in tot een gemiddelde return van $-56,36$ met slechts 91,5% succes. De verklaring volgt uit vergelijking (3): bij hoge α slaat een enkele ϵ -greedy random actie met slechte uitkomst direct door in $Q(s_t, a_t)$ via de bootstrap $Q(s_{t+1}, a_{t+1})$ van diezelfde slechte actie. Q-learning is hier immuun voor, omdat zijn bootstrap altijd $\max_{a'} Q(s_{t+1}, a')$ gebruikt — de slechte actie wordt simpelweg buiten beschouwing gelaten.

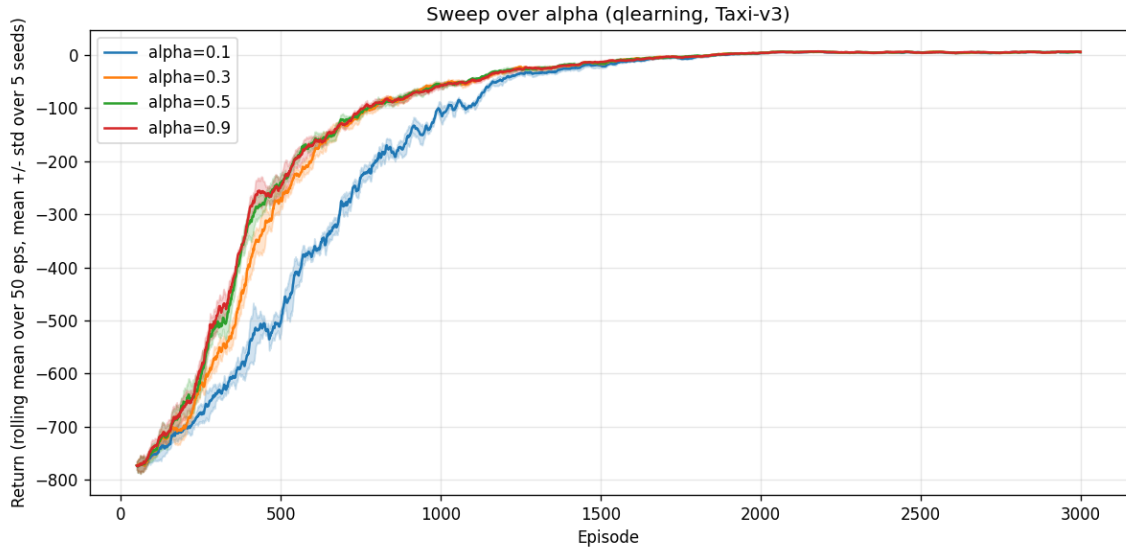
5 Discussie en reflectie

5.1 Vergelijking met de baselines

De grote sprong tussen heuristiek (21% succes) en RL (95–100%) onderstreept waarom RL de juiste paradigmakeuze is voor Taxi: de heuristiek faalt niet omdat zijn optimaliseringscriterium verkeerd is, maar omdat zij een kritisch aspect van de omgeving (muren) niet expliciet modelleert. Een rijkere heuristiek met muurbewustzijn zou ongetwijfeld beter presteren — maar zou ook handmatig voor elk nieuw raster opnieuw geprogrammeerd moeten worden, terwijl Q-learning en SARSA dezelfde implementatie kunnen toepassen op iedere discrete MDP. Dit illustreert de generaliseerbaarheid die RL fundamenteel onderscheidt van rule-based en supervised methoden [1].



Figuur 3: SARSA greedy policy, zelfde acht scenario's als Figuur 2.



Figuur 4: Q-learning, sweep over α . Mean $\pm 1\sigma$ over 5 seeds.

5.2 Q-learning versus SARSA in dit domein

Op een omgeving als Cliff Walking [1, Voorbeeld 6.6] ontstaat een dramatisch verschil tussen on-policy en off-policy leren omdat de optimale policy direct langs een gevaarlijke afgrond loopt. Op Taxi-v3 ontbreekt zo'n “risico-randvoorwaarde”; de optimale policy is unimodaal en SARSA en Q-learning convergeren beide naar (vrijwel) dezelfde policy. Het overgebleven verschil — 5% minder succes voor SARSA en een gevoeliger α -respons — is precies wat de theorie voorspelt: SARSA's on-policy bootstrap incorporeert exploratie-ruis in Q , wat bij hoge α en hoge ε destabiliseert.

5.3 Hyperparameter-bevindingen

Drie inzichten zijn relevant voor toekomstige toepassingen:

1. **Q-learning is robuust voor α en γ .** In een tabulaire setting met een eindige toestandsruimte levert elke redelijke combinatie convergentie naar de optimale Q -functie.

Tabel 2: Gemiddelde return over de laatste 200 episodes per sweep (mean $\pm$ std over 5 seeds).

Sweep	Waarde	Mean return (last 200)
Q-learning, α	0,1	$+5,59 \pm 0,33$
	0,3	$+5,91 \pm 0,21$
	0,5	$+5,53 \pm 0,34$
	0,9	$+5,62 \pm 0,19$
Q-learning, γ	0,8	$+5,70 \pm 0,32$
	0,9	$+5,57 \pm 0,36$
	0,95	$+5,77 \pm 0,36$
	0,99	$+5,53 \pm 0,34$
Q-learning, ε -decay-eps.	200	$+5,30 \pm 0,22$
	1000	$+5,24 \pm 0,30$
	2000	$+5,53 \pm 0,34$
	5000	$-35,27 \pm 1,18$
SARSA, α	0,1	$+4,94 \pm 0,50$
	0,3	$+5,10 \pm 0,26$
	0,5	$+3,07 \pm 1,06$
	0,9	$-56,36 \pm 6,46$

Hyperparameter-tuning van deze twee is dus weinig rendabel.

2. **ε -decay is wel kritisch.** Een te traag schema houdt de gedragspolitiek te lang random; evaluaties tijdens training reflecteren dan eerder de gedragspolitiek dan de geleerde policy.
3. **Algoritme $\times$ hyperparameter is geen productruimte.** Een waarde die voor Q-learning veilig is ($\alpha = 0,9$) is voor SARSA destructief. Tuning moet per algoritme gebeuren.

5.4 Beperkingen

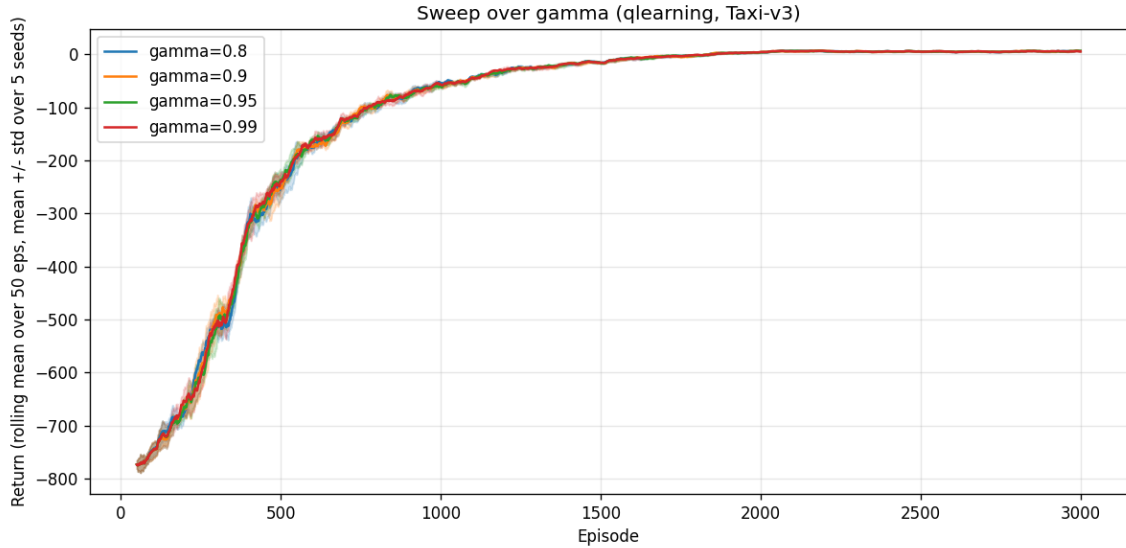
Schaal. Tabulaire methoden vereisen één $Q(s, a)$ -cel per staat-actie-paar. Voor Taxi-v3 ($500 \times 6 = 3000$ cellen) is dit triviaal, maar in elke realistische omgeving met continue of hoog-dimensionale toestanden onhaalbaar. De volgende stap is functie-approximatie via neurale netwerken (Deep Q-Networks, [6]), waar dezelfde Bellman-update wordt toegepast op de gradient van een neuraal netwerk in plaats van een tabel.

Stochasticiteit. Taxi-v3 is een deterministische omgeving: elke actie heeft een vast effect. Toevoeging van stochastische overgangen (zoals de `is_slippery`-modus van CliffWalking) zou de Q-learning vs SARSA contrast verder kunnen versterken en is een natuurlijke uitbreiding voor vervolgwerk.

Sample-efficiëntie. Beide algoritmen vereisen rond de 2000 episodes voor convergentie. Methoden zoals Dyna-Q [1, §8.2] kunnen dit verlagen door ervaringen te “replayen” in een geleerd omgevingsmodel.

6 Conclusie

We hebben Q-learning en SARSA vanaf nul geïmplementeerd en toegepast op de Taxi-v3 omgeving. Q-learning bereikt 100% greedy-succes met een gemiddelde episode-lengte dichtbij het



Figuur 5: Q-learning, sweep over γ .

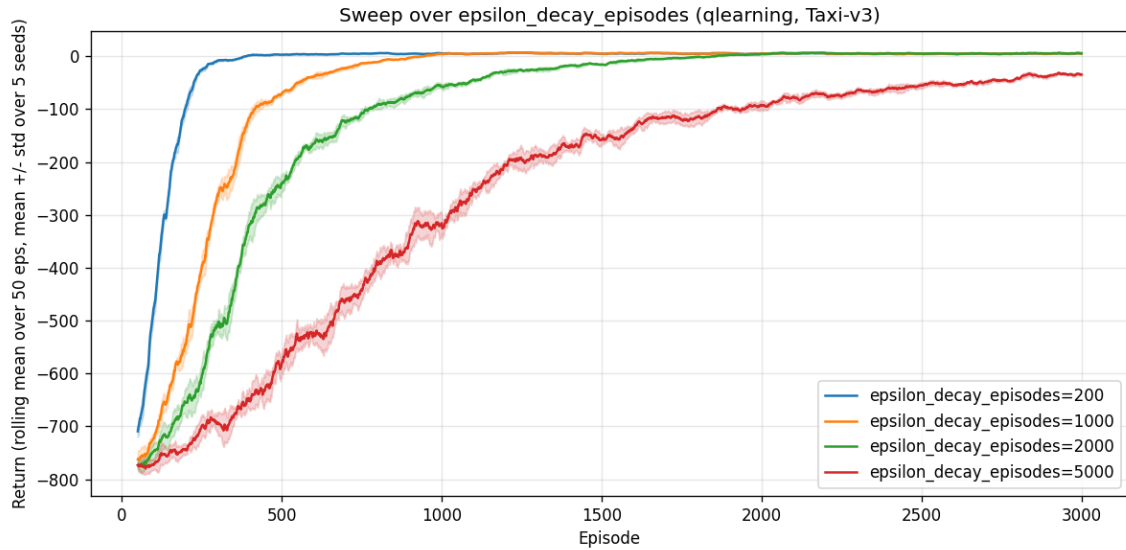
theoretisch minimum; SARSA komt op 95%. Beide algoritmen verslaan zowel een uniform-random baseline als een muur-onbewuste Manhattan-shortest-path heuristiek met ordegrootten. Hyperparameter-sweeps met meerdere seeds tonen aan dat Q-learning robuust is voor α en γ maar gevoelig voor te trage ε -decay, terwijl SARSA dramatisch instort bij hoge α — in lijn met de theoretische voorspelling dat de on-policy bootstrap exploratie-ruis incorporeert in de waardeschatting.

De gekozen omgeving en aanpak hebben ons in staat gesteld de centrale RL-concepten (Markov Decision Process, beloningen, ε -greedy exploratie, on-policy versus off-policy bootstrap, hyperparameter-gevoeligheid) op een werkende implementatie te demonstreren, terwijl de tabulaire schaal van $|S| \cdot |A| = 3000$ tegelijk laat zien waarom function-approximation methoden zoals DQN nodig zijn voor grotere problemen.

Reproduceerbaarheid

Alle code, configuraties en getrainde Q-tabellen zijn beschikbaar in de projectrepository. Trainingen worden gestuurd via YAML-configuraties in `experiments/`; resultaten worden per run weggeschreven naar `results/<run_name>/log.csv`. Elke commando-regelaanroep accepteert `-seed` voor reproduceerbaarheid; de figuren in dit rapport zijn gegenereerd door de scripts `src/plot_results.py`, `src/plot_policy.py` en `src/plot_sweep.py`. De volledige verzameling sweeps is reproduceerbaar met:

```
python -m src.sweep --agent qlearning --param alpha \
    --values 0.1 0.3 0.5 0.9 --seeds 5 --episodes 3000
python -m src.sweep --agent qlearning --param gamma \
    --values 0.8 0.9 0.95 0.99 --seeds 5 --episodes 3000
python -m src.sweep --agent qlearning --param epsilon_decay_episodes \
    --values 200 1000 2000 5000 --seeds 5 --episodes 3000
python -m src.sweep --agent sarsa --param alpha \
    --values 0.1 0.3 0.5 0.9 --seeds 5 --episodes 3000
```

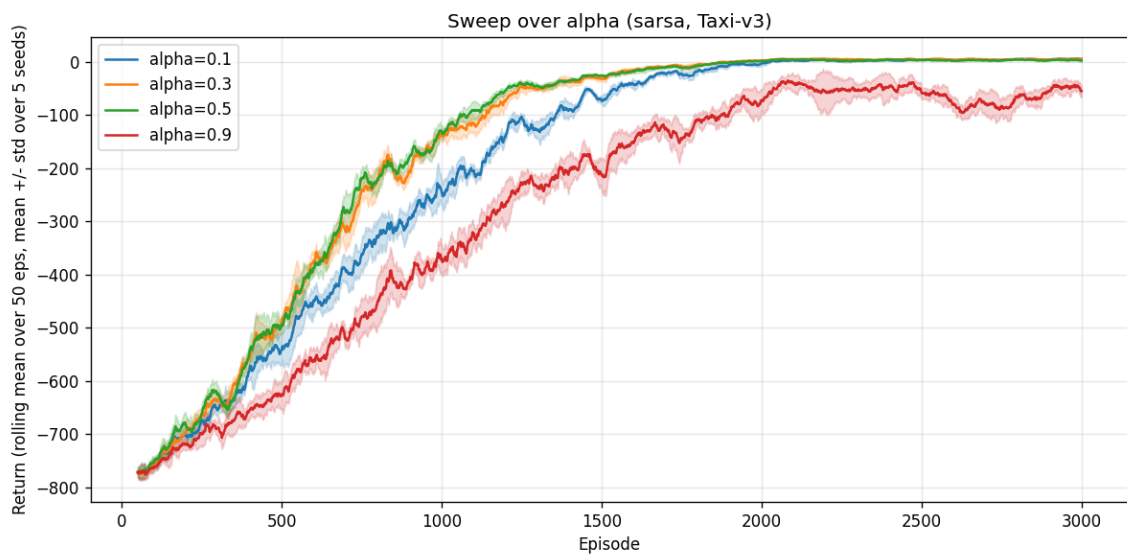
Figuur 6: Q-learning, sweep over de lengte van het ε -decay-schema. Bij 5000 episodes blijft ε tijdens de gehele training te hoog, met een dramatisch lagere trainings-return als gevolg.

Gebruik van Generative AI

Bij het schrijven van deze rapportage en de implementatie is gebruik gemaakt van Anthropic's Claude (model claude-opus-4-7, mei 2026) voor sparring over de structuur van het rapport, debugging van de trainingspipeline en het opstellen van de LaTeX-skelet. Alle algoritmische beslissingen, hyperparameter-keuzes en interpretaties zijn door de auteurs geverifieerd tegen de geciteerde bronnen.

Referenties

- [1] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [2] Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292. <https://doi.org/10.1007/BF00992698>
- [3] Rummery, G. A., & Niranjan, M. (1994). *On-line Q-learning using connectionist systems* (Technical Report CUED/F-INFENG/TR 166). University of Cambridge, Department of Engineering.
- [4] Dietterich, T. G. (2000). Hierarchical reinforcement learning with the MAXQ value function decomposition. *Journal of Artificial Intelligence Research*, 13, 227–303. <https://doi.org/10.1613/jair.639>
- [5] Towers, M., Kwiatkowski, A., Terry, J., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. *arXiv:2407.17032*. <https://arxiv.org/abs/2407.17032>
- [6] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>



Figuur 7: SARSA, sweep over α . In tegenstelling tot Q-learning (Fig. 4) stort SARSA bij hoge α in.